

Section 8.4: Serverless Architecture

Module 8 of 8 · FaaS, Event-Driven Patterns & Cost Modelling

Executive Overview

Serverless computing abstracts infrastructure management away from developers, but introduces its own challenges: cold start latency, statelessness, execution limits, and a non-obvious cost model. Understanding when serverless helps versus hurts — and how to mitigate its weaknesses — is essential for cloud architecture interviews.

Part 1: Function-as-a-Service (FaaS) Execution Model

1.1 Container Lifecycle

Every Lambda/Cloud Function invocation runs inside a container. That container either already exists (warm) or must be created (cold).

Cold Start Path (first invocation or after scale-out):

1. Container allocation	~5-50 ms	
2. Runtime init (JVM/Node)	~20-200 ms	
3. Dependency loading	~50-500 ms	← biggest variable
4. Handler init code	~10-100 ms	
5. First invocation	~5-50 ms	
Total cold start:	~90 ms - 900 ms	

Warm Invocation Path (subsequent invocations on same container):

Container already running	
Runtime already initialized	
Handler init already done	
Invocation only:	~5-50 ms

Cold start by runtime (AWS Lambda, typical):

Runtime	Typical Cold Start	Why
Node.js 20	80–200 ms	Fast V8 init; small stdlib
Python 3.12	100–300 ms	Medium interpreter init
Go (custom runtime)	50–150 ms	Compiled binary; no GC init
Java 21 (JVM)	500–1500 ms	JVM warmup; class loading
Java 21 + SnapStart	50–200 ms	Snapshot restores post-init state
.NET 8	200–600 ms	CLR initialisation

1.2 Cold Start Mitigation — Comparison

Technique	Eliminates Cold Start?	Cost Impact	Complexity	Use When
Provisioned Concurrency	Yes (fully)	High — always-on billing	Low	P99 latency critical; user-facing
SnapStart (Java only)	Mostly (90% faster)	None extra	Low	Java on Lambda with tight latency
Scheduled Warm-up Ping	Partially (race condition)	Very low (~\$1/month)	Low	Dev/test; non-critical paths
Minimize package size	No — reduces init time	None	Medium	Always; default practice
ARM (Graviton2)	No — improves it	Saves 20% cost too	None	Always; default to arm64
Keep-alive HTTP	No	None	None	Connection reuse; doesn't help cold start

```
# Provisioned Concurrency in AWS SAM
Resources:
  OrderProcessor:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: nodejs20.x
      Architectures: [arm64]           # Graviton2 — cheaper + faster
      AutoPublishAlias: live
```

```
ProvisionedConcurrencyConfig:
  ProvisionedConcurrentExecutions: 10 # Always-warm containers
```

1.3 Concurrency Model & Scaling

Lambda Concurrency (per account, per region):

Default limit: 1,000 concurrent executions
Burst limit: 500-3,000 immediate (varies by region)
Scaling rate: +500 per minute after burst

If limit exceeded: Throttling (HTTP 429) – requests queued or dropped

Reserved Concurrency:

- Guarantees N concurrency slots for one function
- Prevents other functions from taking those slots
- Also caps maximum concurrency for that function

Formula: Concurrency = (Invocations/sec) × (Duration in seconds)

Example: 100 req/s × 0.5s avg = 50 concurrent executions needed

Part 2: Serverless Event-Driven Patterns

2.1 Common Trigger Patterns

Trigger	Delivery	Use Case	Gotcha
API Gateway	Synchronous	REST/HTTP APIs	29s timeout hard limit
S3 Event	Async, at-least-once	File processing	Possible duplicate invocation
DynamoDB Streams	Async, ordered per shard	Change data capture, materialised views	Shard capacity limit
SQS	Async, at-least-once	Work queues, fan-out	Visibility timeout must exceed Lambda timeout
SNS	Async, fan-out	Pub-sub notifications	No ordering guarantee
EventBridge	Async, filtered	Scheduled jobs, event routing	5-minute minimum schedule
Kinesis	Async, ordered		

Trigger	Delivery	Use Case	Gotcha
		High-throughput streaming	Checkpoint management needed

2.2 Step Functions — Orchestrating Multi-Step Workflows

Prefer Step Functions over chained Lambda calls when you need retries, error handling, parallelism, or visibility.

```
{
  "Comment": "Order processing workflow",
  "StartAt": "ValidateOrder",
  "States": {
    "ValidateOrder": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123:function:validate-order",
      "Next": "ReserveInventory",
      "Catch": [{ "ErrorEquals": ["ValidationError"], "Next": "RejectOrder" }]
    },
    "ReserveInventory": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123:function:reserve-inventory",
      "Retry": [{ "ErrorEquals": ["States.TaskFailed"], "MaxAttempts": 3,
"BackoffRate": 2 }],
      "Next": "ProcessPayment",
      "Catch": [{ "ErrorEquals": ["States.ALL"], "Next": "ReleaseInventory" }]
    },
    "ProcessPayment": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123:function:process-payment",
      "Next": "ConfirmOrder",
      "Catch": [{ "ErrorEquals": ["PaymentFailed"], "Next": "ReleaseInventory" }]
    },
    "ConfirmOrder": { "Type": "Task", "Resource": "...", "End": true },
    "ReleaseInventory": { "Type": "Task", "Resource": "...", "Next":
"RejectOrder" },
    "RejectOrder": { "Type": "Task", "Resource": "...", "End": true }
  }
}
```

Lambda chaining vs Step Functions:

Concern	Lambda Chaining	Step Functions
Retry logic	Must implement manually in each function	Built-in with exponential backoff
Error handling		

Concern	Lambda Chaining	Step Functions
	Try/catch in code; fragile across hops	Catch blocks in state machine definition
Visibility	No built-in execution history	Full execution history in AWS console
Branching	Conditional code inside functions	Choice states in definition
Cost	No orchestration cost	\$0.025 per 1000 state transitions
Max duration	15 min per function	Up to 1 year (Express: 5 min)

2.3 Stateless Constraint — Right Patterns

```

# ❌ WRONG — in-memory state not reliable across invocations
counter = 0

def handler(event, context):
    global counter
    counter += 1
    return {"count": counter} # Different containers = different counters

# ✅ CORRECT — externalise all state
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Counters')

def handler(event, context):
    response = table.update_item(
        Key={'id': 'global'},
        UpdateExpression='ADD #count :inc',
        ExpressionAttributeNames={'#count': 'count'},
        ExpressionAttributeValues={':inc': 1},
        ReturnValues='UPDATED_NEW'
    )
    return {"count": int(response['Attributes']['count'])}

```

Part 3: When NOT to Use Serverless

3.1 Decision Framework

Criteria	Serverless Wins	Containers/EC2 Win
Traffic pattern	Spiky, intermittent, unpredictable	Steady, predictable 24/7
Latency requirement	> 100ms P99 acceptable	< 50ms P99 required
Max execution time	< 15 minutes	Long-running (video encoding, ML training)
State management	Stateless or simple external state	Complex in-memory state, WebSockets
Cost at scale	< ~3M invocations/day	> ~3M invocations/day
Compliance	Standard requirements	Strict data residency, dedicated hardware

3.2 Cost Crossover Analysis

Scenario: Web API serving 1M invocations/day, 100ms avg duration, 256MB memory

Lambda cost:

Requests: $1,000,000 \times \$0.20/\text{million} = \$0.20/\text{day}$

Duration: $1,000,000 \times 0.1s \times 256\text{MB}/1024 \times \$0.0000166667/\text{GB-s} = \$0.43/\text{day}$

Total Lambda: ~\$0.63/day (~\$19/month)

EC2 m5.large (2 vCPU, 8GB):

On-demand: \$70/month

1-year reserved: \$44/month

Break-even: Lambda more expensive than reserved EC2 at ~2.3M invocations/day

With Provisioned Concurrency (10 units, 100ms avg):

PC cost: $10 \times \$0.0000046875/\text{GB-s} \times 3600 \times 24 = \$20.25/\text{day extra}$

Total with PC: ~\$21/day (~\$630/month) – clearly use EC2

Part 4: Interview Design Problems

Problem 1 — Image Processing Pipeline

Question: Design a serverless system that resizes uploaded images to three sizes (thumbnail, medium, large) and stores them in S3. Handle failures gracefully. What's your SLA for processing latency?

Solution:

Architecture:

```
User → API Gateway → Upload Lambda → S3 (originals bucket)
S3 event → SQS → Resize Lambda (×N concurrent)
                → S3 (processed bucket)
                → DynamoDB (image metadata)
```

```
# Resize Lambda — triggered by SQS
import boto3, json
from PIL import Image
from io import BytesIO

s3 = boto3.client('s3')
dynamo = boto3.resource('dynamodb').Table('ImageMetadata')

SIZES = {'thumbnail': (150, 150), 'medium': (800, 600), 'large': (1920, 1080)}

def handler(event, context):
    for record in event['Records']:
        s3_event = json.loads(record['body'])
        bucket = s3_event['bucket']['name']
        key = s3_event['object']['key']

        # Download original
        response = s3.get_object(Bucket=bucket, Key=key)
        image = Image.open(BytesIO(response['Body'].read()))

        results = {}
        for size_name, dimensions in SIZES.items():
            resized = image.copy()
            resized.thumbnail(dimensions, Image.LANCZOS)

            buffer = BytesIO()
            resized.save(buffer, format=image.format, quality=85)
            buffer.seek(0)

            dest_key = f"processed/{size_name}/{key}"
            s3.put_object(Bucket='processed-images', Key=dest_key, Body=buffer)
            results[size_name] = dest_key

        dynamo.put_item(Item={'imageId': key, 'status': 'processed', 'variants':
```

```
results}))

# Error handling: SQS visibility timeout = 5 min > Lambda 3 min timeout
# Failed messages go to DLQ after 3 retries
# DLQ alarm triggers PagerDuty
```

SLA analysis: - S3 upload triggers SQS within ~1s - Lambda cold start: ~200ms (Node) → use provisioned concurrency for P99 < 500ms - Processing 5MB image: ~2s - Total: P50 ~3s, P99 ~5s (with provisioned), P99 ~6s (without)

Problem 2 — Serverless API with Traffic Spikes

Question: A marketing campaign will send 50,000 simultaneous requests to your Lambda-backed API. How do you prevent throttling and protect your database?

Solution:

```
# Pattern: SQS as buffer between API Gateway and Lambda

# Layer 1: API Gateway → SQS (handles burst absorption)
# API Gateway: no concurrency limit; SQS absorbs the spike

# Layer 2: SQS → Lambda (controlled consumption)
# Lambda: reserved concurrency = 200 (protects downstream)

# Layer 3: Lambda → RDS Proxy (connection pooling)
# RDS Proxy: pools connections, prevents 50K simultaneous DB connections

# Lambda with RDS Proxy
import pymysql
import os

# Connection reuse across warm invocations
conn = None

def get_connection():
    global conn
    if conn is None or not conn.open:
        conn = pymysql.connect(
            host=os.environ['RDS_PROXY_HOST'], # Proxy endpoint
            user=os.environ['DB_USER'],
            password=os.environ['DB_PASSWORD'],
            db='mydb',
            connect_timeout=5
        )
    return conn

def handler(event, context):
    db = get_connection()
```



```
# Process SQS message
for record in event['Records']:
    body = json.loads(record['body'])
    with db.cursor() as cursor:
        cursor.execute("INSERT INTO events ...", body)
    db.commit()
```

Throttling calculation:

50,000 simultaneous requests
 Without SQS: API Gateway → Lambda, needs 50,000 concurrent executions
 → Account limit is 1,000 → 49,000 throttled immediately

With SQS buffer:
 SQS accepts all 50,000 messages instantly (no limit)
 Lambda processes at reserved concurrency = 200
 Processing time: $50,000 / 200 / 10 \text{ RPS per Lambda} = 25 \text{ seconds}$
 All messages processed; zero throttling errors to caller
 (Caller receives 202 Accepted immediately)

Problem 3 — Hybrid Serverless/Container Architecture

Question: You have a checkout API (must be fast, low latency) and a recommendation engine (batch, complex ML inference). How do you architect both, and where does each run?

Solution:

Checkout API (user-facing, latency-critical):

- EC2 / ECS Fargate containers
- Always-on, load-balanced
- P99 < 50ms requirement
- Provisioned Concurrency Lambda acceptable if P99 < 100ms acceptable

Recommendation Engine (batch, ML inference):

- Lambda triggered by scheduled EventBridge rule (every 30 min)
- Or SageMaker batch transform for large models
- 15-minute Lambda limit fits 10M user batch in 12 min
- Results written to DynamoDB; checkout API reads pre-computed recommendations

Architecture:

User → API Gateway → ECS Fargate (checkout) → DynamoDB (reads reco)
 EventBridge (every 30 min) → Lambda (reco engine) → SageMaker → DynamoDB (writes reco)

Cost estimate for 1M users, 100K daily active:

Checkout API (ECS Fargate):

2 tasks × m5.large equivalent = \$70/month

Autoscale to 10 tasks at peak = \$350/month peak

Recommendation Lambda (30-min schedule):

48 invocations/day × 12 min × 3GB memory

= $48 \times 720s \times 3GB \times \$0.0000166667 = \$1.73/\text{day} = \sim \$52/\text{month}$

Total: ~\$400-450/month vs pure Lambda ~\$1,200/month at this volume

Quick Reference

Concept	Value
Lambda default account concurrency limit	1,000 concurrent
Lambda maximum timeout	15 minutes
API Gateway maximum timeout	29 seconds
Lambda cold start range (Node.js)	80–300 ms
Lambda cold start range (Java/JVM)	500–1,500 ms
Break-even vs EC2 reserved	~2–3M invocations/day
SQS visibility timeout rule	Must exceed Lambda timeout
Provisioned Concurrency billing	Per GB-second, always-on